MDPI

Article

Role-Symmetric EAP-EDHOC with Hybrid Post-Quantum Cryptography for 5G Secondary Authentication and Application-Server Access

Adi Panca Saputra Iskandar [1,4], SeongHan Shin [2], Yongho Ko [1], I Wayan Adi Juliawan Pawana [1,4], Hoseok Kwon [1] and Ilsun You [1,3,*]

[1] Department of Cybersecurity, Kookmin University, Seoul 02707, Republic of Korea; adipancaiskandar@gmail.com (A.P.S.I.); koyh0911@kookmin.ac.kr (Y.K.); adijuliawan@kookmin.ac.kr (I.W.A.J.P.)
[2] Cyber Physical Security Research Center, National Institute of Advanced Industrial Science and Technology (AIST), Tokyo 135-0064, Japan; seonghan.shin@aist.go.jp
[3] KMU Global Research Center for ICT Convergence Security, Kookmin University, Seoul 02707, Republic of Korea
[4] Department of Electrical Engineering, Udayana University, Badung, Bali 80361, Indonesia
* Correspondence: isyou@kookmin.ac.kr

Abstract

5G secondary authentication lets an external data network (DN) authenticate user equipment (UE) through the Extensible Authentication Protocol (EAP), but existing methods such as EAP-TLS are heavyweight and rely on elliptic-curve cryptography that a quantum adversary can break. Moving to post-quantum key encapsulation also breaks the role symmetry of authenticated key exchange, because a key encapsulation mechanism (KEM) implicitly authenticates only the decapsulating party. We propose a role-symmetric hybrid post-quantum EAP-EDHOC (Ephemeral Diffie–Hellman Over COSE) protocol that restores this symmetry: the UE and the DN authentication server each receive exactly one X-Wing (X25519 + ML-KEM-768) encapsulation to a one-time static key, each authenticate with a detached ML-DSA-44 signature, and each rotate and erase their key after every run. The mirrored rotation gives both parties forward secrecy and post-compromise security, and the independent signatures give Key-Compromise Impersonation (KCI) resistance in both directions. The exported Extended Master Session Key bootstraps application-layer keys for login-free server access. We model nine security requirements in ProVerif for seven methods, including a negative control without key erasure. An implementation on Open5GS, UERANSIM, and FreeRADIUS shows that the symmetry extends to the traffic: of the 12,228-byte handshake, the UE sends 6,116 bytes and the server 6,112 bytes. The cost, in contrast, is asymmetric and lies in transport fragmentation rather than in cryptography.

Keywords: symmetric mutual authentication; asymmetric cryptography; hybrid post-quantum cryptography; EDHOC; 5G secondary authentication; X-Wing KEM; ML-DSA; formal verification

Received:
Revised:
Accepted:
Published:

Copyright: © 2026 by the authors. Submitted to *Symmetry* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

A 5G operator serves many network slices and external data networks, such as enhanced mobile broadband (eMBB), ultra-reliable low-latency communication (URLLC), massive machine-type communication (mMTC), enterprise and industrial networks, and Multi-access Edge Computing (MEC). Primary authentication through 5G Authentication

and Key Agreement (5G-AKA) or EAP-AKA′ only proves that the subscriber is legitimate for the operator and does not grant access to these services [1,2]. It is also not a sufficient trust anchor on its own: formal analyses of 5G-AKA have revealed protocol-level weaknesses [3] and a privacy attack that survives the move from 4G to 5G [4]. 5G therefore defines *secondary authentication* based on the Extensible Authentication Protocol (EAP) [1,5], in which the user equipment (UE) authenticates to a data network (DN) Authentication, Authorization, and Accounting (DN-AAA) server owned by the service provider, so that connectivity is authenticated by the operator while service-level authorization stays with the service owner. Dedicated UE–DN authentication frameworks [6] and analyses of the EAP-TLS profile [1,7] show the growing attention paid to this step.

Figure 1 places the flow in the 3GPP architecture. After primary authentication between the Access and Mobility Management Function (AMF) and the Authentication Server Function (AUSF), the UE reaches the DN-AAA server over the Session Management Function (SMF)/User Plane Function (UPF) path to run secondary authentication; the DN-AAA then bootstraps keys toward an application server (AS), using the Extended Master Session Key (EMSK), so the UE reaches the AS without a further login.

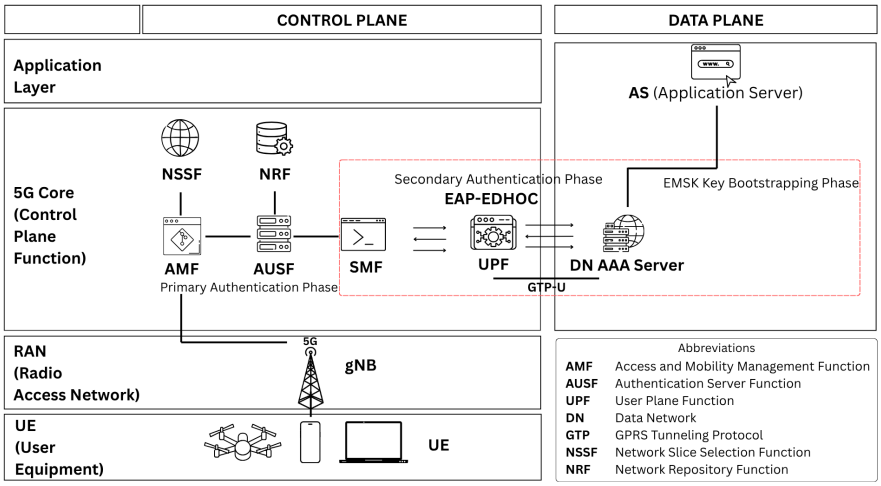


Figure 1. 5G secondary authentication phase and Extended Master Session Key (EMSK) key-bootstrapping phase. AMF: Access and Mobility Management Function; AUSF: Authentication Server Function; SMF: Session Management Function; UPF: User Plane Function; DN-AAA: data network Authentication, Authorization, and Accounting server.

EAP-TLS (Transport Layer Security) [7] is costly for constrained Internet of Things (IoT) and industrial devices because of certificate exchange and the TLS handshake. EDHOC (Ephemeral Diffie–Hellman Over COSE) [8,9], a lightweight authenticated key exchange based on the Concise Binary Object Representation (CBOR) and CBOR Object Signing and Encryption (COSE), has been carried into EAP by the EAP-EDHOC draft [10] and is markedly more efficient than Datagram TLS (DTLS) 1.3 [11,12]. EDHOC, however, still relies on elliptic-curve Diffie–Hellman, which Shor's algorithm breaks [13] and which is exposed to *harvest-now, decrypt-later* attacks. Hybrid post-quantum cryptography (PQC) is the recommended transition path because it keeps interoperability with classical systems [14–18]; X-Wing [19,20] is a compact hybrid key encapsulation mechanism (KEM) that combines X25519 with ML-KEM-768. Real hybrid-PQC deployments of EAP-EDHOC in an open-source 5G system, and the effect of PQC payloads on Remote Authentication Dial-In User Service (RADIUS) encapsulation [21,22], remain largely unexplored [23–26].

Symmetry perspective. Authenticated key exchange is governed by a symmetry between its two parties. Diffie–Hellman is inherently symmetric, since both sides con-

tribute a share and derive the same secret, but a KEM is asymmetric: only the holder of the decapsulation key is implicitly authenticated. EDHOC's four methods also mix symmetric (SIG-SIG, STAT-STAT) and asymmetric, mirror-image (SIG-STAT, STAT-SIG) role assignments. Replacing Diffie–Hellman with a KEM therefore breaks the role symmetry unless the protocol restores it. Our design restores it explicitly: each party is the target of exactly one X-Wing encapsulation to a one-time static key, each party is authenticated by its own detached ML-DSA-44 signature, and each party rotates its static key after every run, which also replaces the ephemeral key pair as the source of forward secrecy. At the same time, we show that the protocol's *costs* are strongly asymmetric, cheap in computation and expensive in transport, which determines where deployment effort should go.

The contributions of this work are as follows:

1. A role-symmetric hybrid PQC EAP-EDHOC protocol for 5G secondary authentication, built on X-Wing with a SIGMA-style ML-DSA-44 gate that resists Key-Compromise Impersonation (KCI) and a symmetric ~~key rotation that provides~~ rotation of one-time static keys that provides both forward secrecy and post-compromise security without an ephemeral key pair.
2. A ProVerif [27,28] analysis of nine security requirements, comparing the proposal with the four classical EDHOC methods and two EAP-TLS baselines, with a negative-control model showing that forward secrecy rests on key erasure.
3. An end-to-end implementation on Open5GS [29], UERANSIM [30], and FreeRADIUS [31], with open-source code [32], including EMSK-based key bootstrapping of Object Security for Constrained RESTful Environments (OSCORE) for login-free application-server access.
4. A performance evaluation of computation, communication, EAP/RADIUS fragmentation, and packet-loss behaviour that quantifies the asymmetry between cryptographic and transport cost.

2. Related Work

Security analysis of classical EDHOC. RFC 9528 has been analysed symbolically by Jacomme et al. [33] (all four methods), Norrman et al. [34], and Kim et al. [35], and computationally by Günther and Mukendi [36], Cremers et al. [37], Cottier and Pointcheval [38], and Ferreira [39]; Pérez et al. [12] survey this work. A recurring finding is that the MAC-based methods (T1–T3) are vulnerable to KCI once the static Diffie–Hellman key leaks [36,37]. These works analyse EDHOC as specified, which remains purely classical.

Post-quantum and hybrid EDHOC. Fraile et al. [23] propose PQ-EDHOC, a fully post-quantum variant using NIST KEMs and signatures, evaluated with microbenchmarks on constrained hardware. A companion work [24] and IETF draft [25] add a signature-free, KEM-based authentication method. Shin et al. [26] embed an ephemeral PQC KEM into EDHOC and omit the optional Message 4, and in follow-up work [40] they strengthen this hybrid EDHOC design toward post-quantum security against KCI. The present paper builds on that line of work and further hardens KCI resistance with a detached ML-DSA-44 gate that is independent of the KEM static keys. None of these proposals target 5G secondary authentication or integrate with EAP/RADIUS, and none combines KCI resistance with post-compromise key rotation.

5G secondary authentication. Gong et al. [6] propose new network functions for UE–DN mutual authentication, and the EAP-TLS profile of TS 33.501 Annex B has been formally analysed [1,7]. These works do not consider PQC or the EAP-over-RADIUS fragmentation that, as we show in Section 6, becomes the bottleneck once PQC payloads are introduced. No prior work combines hybrid PQC, a KCI-resistant gate independent of

the KEM static key, formal verification, a live 5G implementation, and EMSK-based key bootstrapping.

Table 1 classifies the seven evaluated methods by the mechanism that authenticates each party. Read row by row, it exposes the role symmetry of each design: Method 0 (SIG-SIG) and Method 3 (STAT-STAT) are role-symmetric, while Methods 1 and 2 are mirror images of each other, each authenticating one party by signature and the other by a static-DH MAC. The static-DH MAC is the mechanism shown to be KCI-vulnerable [36,37]. EAP-TLS is role-symmetric, with entity authentication provided by the `CertificateVerify` signature; its HMAC `Finished` message is a universal key-confirmation step rather than a separate authentication mechanism. Only three methods are post-quantum, and only the proposed method pairs role symmetry with a signature that is both post-quantum and detached from the key-exchange static key. Although the proposal and EAP-TLS Hybrid PQC share the same primitives, they differ in how authentication is framed, which Section 6 shows matters more for latency than the primitives themselves.

Table 1. Cryptographic building blocks of the seven evaluated methods: the signature- or message authentication code (MAC)-based mechanism authenticating the initiator and the responder. SIG: signature; STAT: static Diffie–Hellman key; DH: Diffie–Hellman; HKDF: HMAC-based key derivation function; PQC: post-quantum cryptography.

Method	Initiator authentication	Responder authentication	PQC
EAP-EDHOC Method 0 (SIG-SIG) [8,10]	Signature (Ed25519)	Signature (Ed25519)	No
EAP-EDHOC Method 1 (SIG-STAT) [8,10]	Signature (Ed25519)	Static-DH MAC (HKDF/HMAC-SHA256)	No
EAP-EDHOC Method 2 (STAT-SIG) [8,10]	Static-DH MAC (HKDF/HMAC-SHA256)	Signature (Ed25519)	No
EAP-EDHOC Method 3 (STAT-STAT) [8,10]	Static-DH MAC (HKDF/HMAC-SHA256)	Static-DH MAC (HKDF/HMAC-SHA256)	No
EAP-EDHOC Hybrid PQC (Section 4, over [10])	**Signature (ML-DSA-44, detached SIGMA gate)**	**Signature (ML-DSA-44, detached SIGMA gate)**	**Yes (hybrid)**
EAP-TLS Classic [7]	Signature (Ed25519, X.509 `CertificateVerify`) + HMAC `Finished`	Signature (Ed25519, X.509 `CertificateVerify`) + HMAC `Finished`	No
EAP-TLS Hybrid PQC [7,41]	Signature (ML-DSA-44, X.509 `CertificateVerify`) + HMAC `Finished`	Signature (ML-DSA-44, X.509 `CertificateVerify`) + HMAC `Finished`	Yes (hybrid)

3. Preliminaries

3.1. 5G Secondary Authentication

EAP [5] defines three roles: the *peer* (the UE), the *authenticator*, which relays messages without understanding the method (the SMF), and the *EAP server*, which runs the method and decides (the DN-AAA) [1,2]. EAP frames travel over the SMF–UPF–DN-AAA path of Figure 1 and are carried in RADIUS between the SMF and the DN-AAA [21,22]. Unlike primary authentication, which the AUSF terminates inside the operator's domain, secondary authentication is terminated by a DN-AAA that the operator does not control and runs only after the PDU session is established [1]. A successful method exports the Master Session Key (MSK) and the Extended MSK (EMSK) [5,42]. The MSK protects the UE–DN session; the EMSK never leaves the peer and the server and serves as a root for cryptographically separated keys through RFC 5295's Usage-Specific Root Keys (USRK). We use it to bootstrap an OSCORE [43] context with an application server without any further login.

3.2. X-Wing Hybrid KEM

X-Wing [19,20] combines X25519 with ML-KEM-768 (FIPS 203) [44,45], following the IETF hybrid terminology [17] and the rationale of hybrid KEMs for authenticated key exchange [18,46]. The combined secret stays secure as long as *at least one* component is

unbroken. Algorithm 1 summarises its three operations. The public key $pk = ek_{\mathrm{PQ}} \,\|\, pk_X$ is $1184 + 32$ bytes. Encapsulation runs ML-KEM-768 and an ephemeral X25519 exchange and merges both shares with a single SHA3-256 *combiner* that also binds both public shares, which blocks re-encapsulation and mismatch attacks [37,47]; decapsulation recomputes the same 32-byte secret.

X-Wing is IND-CCA secure under standard assumptions and, by exploiting ML-KEM's ciphertext binding, needs weaker assumptions than generic combiners [19]. Unlike the negotiated combiners of hybrid TLS 1.3 [41], IKEv2 [48], or composite ML-KEM [18,49], it is a fixed instantiation without runtime negotiation, in line with deployed hybrids such as PQXDH [50,51] and post-quantum Noise [47], and compact enough for COSE/CBOR. Note that a KEM is asymmetric by construction: the encapsulator learns nothing about the decapsulator's long-term key, and only the decapsulator is implicitly authenticated. Section 4.4 shows how the protocol compensates for this.

Algorithm 1 X-Wing hybrid KEM: KeyGen(), Encaps(pk), and Decaps(ct, sk)

KeyGen():
  $(dk_{\mathrm{PQ}}, ek_{\mathrm{PQ}}) \leftarrow$ ML-KEM-768.KeyGen(); $(sk_X, pk_X) \leftarrow$ X25519.KeyGen()
  return $pk \leftarrow ek_{\mathrm{PQ}} \,\|\, pk_X$ $(1184 + 32 = 1216$ bytes), $sk \leftarrow dk_{\mathrm{PQ}} \,\|\, sk_X$
Encaps($pk = ek_{\mathrm{PQ}} \,\|\, pk_X$):
  $(ss_{\mathrm{PQ}}, ct_{\mathrm{PQ}}) \leftarrow$ ML-KEM-768.Encaps(ek_{PQ}); $(sk_e, pk_e) \leftarrow$ X25519.KeyGen()
  $ss_X \leftarrow$ X25519(sk_e, pk_X)
  $ss \leftarrow$ SHA3-256$\big(ss_{\mathrm{PQ}} \,\|\, ss_X \,\|\, pk_e \,\|\, pk_X \,\|\, \texttt{label}_{\text{X-Wing}}\big)$ ▷ combiner
  return $(ss,\ ct \leftarrow ct_{\mathrm{PQ}} \,\|\, pk_e)$
Decaps($ct = ct_{\mathrm{PQ}} \,\|\, pk_e$, $sk = dk_{\mathrm{PQ}} \,\|\, sk_X$):
  $ss_{\mathrm{PQ}} \leftarrow$ ML-KEM-768.Decaps($ct_{\mathrm{PQ}}, dk_{\mathrm{PQ}}$); $ss_X \leftarrow$ X25519(sk_X, pk_e)
  return $ss \leftarrow$ SHA3-256$\big(ss_{\mathrm{PQ}} \,\|\, ss_X \,\|\, pk_e \,\|\, pk_X \,\|\, \texttt{label}_{\text{X-Wing}}\big)$

4. Proposed EAP-EDHOC Hybrid PQC Protocol

4.1. Assumptions and Design Rationale

In a provisioning phase (Algorithm 2), the UE (Initiator) and the DN-AAA (Responder) each generate a long-term X-Wing key pair and a long-term ML-DSA-44 signing key, and each holds the peer's long-term X-Wing public key. The peer's verification key (VK_R, VK_I) is conveyed in `Plaintext2`/`Plaintext3` and bound to the transcript by the corresponding signature. EAP-EDHOC messages are carried over EAP [5] and RADIUS [21,22] between the pass-through authenticator (SMF/UPF) and the DN-AAA, following TS 33.501 [1]. The Dolev–Yao adversary [52] controls the network.

The X-Wing static keys are *one-time* keys. Each party generates its next key pair during a run, announces the public key in EAD_3 or EAD_4, and erases the old secret key once the run completes, so every static key pair serves exactly one session. The DN-AAA therefore keeps one record per UE, holding (sk_B, pk_B) for that UE together with (pk_A, VK_I). The DN-AAA selects this record from the EAP identity of the session, which reaches it in the RADIUS Access-Request before `Message1`, so no extra field is added to the EDHOC messages.

The protocol follows the SIGMA (SIGn-and-MAc) design [53]: X-Wing provides confidentiality and key agreement, while detached ML-DSA-44 signatures [54] are the sole source of entity authentication. Because the signing keys are independent of the X-Wing static keys, leaking a KEM static key does not enable impersonation; this closes the KCI gap of a MAC-only Method 4 variant [36,37]. We choose ML-DSA-44 (NIST category 2) rather than ML-DSA-65 to limit fragmentation: its 1312-byte key and 2420-byte signature are markedly smaller, and its margin still exceeds that of the Ed25519 signatures of T0–T3. Being nominally below the category-3 strength of ML-KEM-768, this is a deliberate size/security trade-off.

Algorithm 2 Long-term key generation (provisioning)

1: **At UE:** $(sk_{\mathsf{A}}, pk_{\mathsf{A}}) \leftarrow$ X-WING.KeyGen(); $(sk_{\mathsf{SIG,I}}, VK_I) \leftarrow$ ML-DSA-44.KeyGen()
2: **At DN-AAA:** $(sk_{\mathsf{B}}, pk_{\mathsf{B}}) \leftarrow$ X-WING.KeyGen(); $(sk_{\mathsf{SIG,R}}, VK_R) \leftarrow$ ML-DSA-44.KeyGen()
3: **return** $(pk_{\mathsf{A}}, sk_{\mathsf{A}}, sk_{\mathsf{SIG,I}}, VK_I)$, $(pk_{\mathsf{B}}, sk_{\mathsf{B}}, sk_{\mathsf{SIG,R}}, VK_R)$

4.2. Handshake

Figure 2 shows the exchange inside the 3GPP secondary-authentication flow, with the SMF as EAP authenticator and the DN-AAA as EAP server reached over RADIUS.

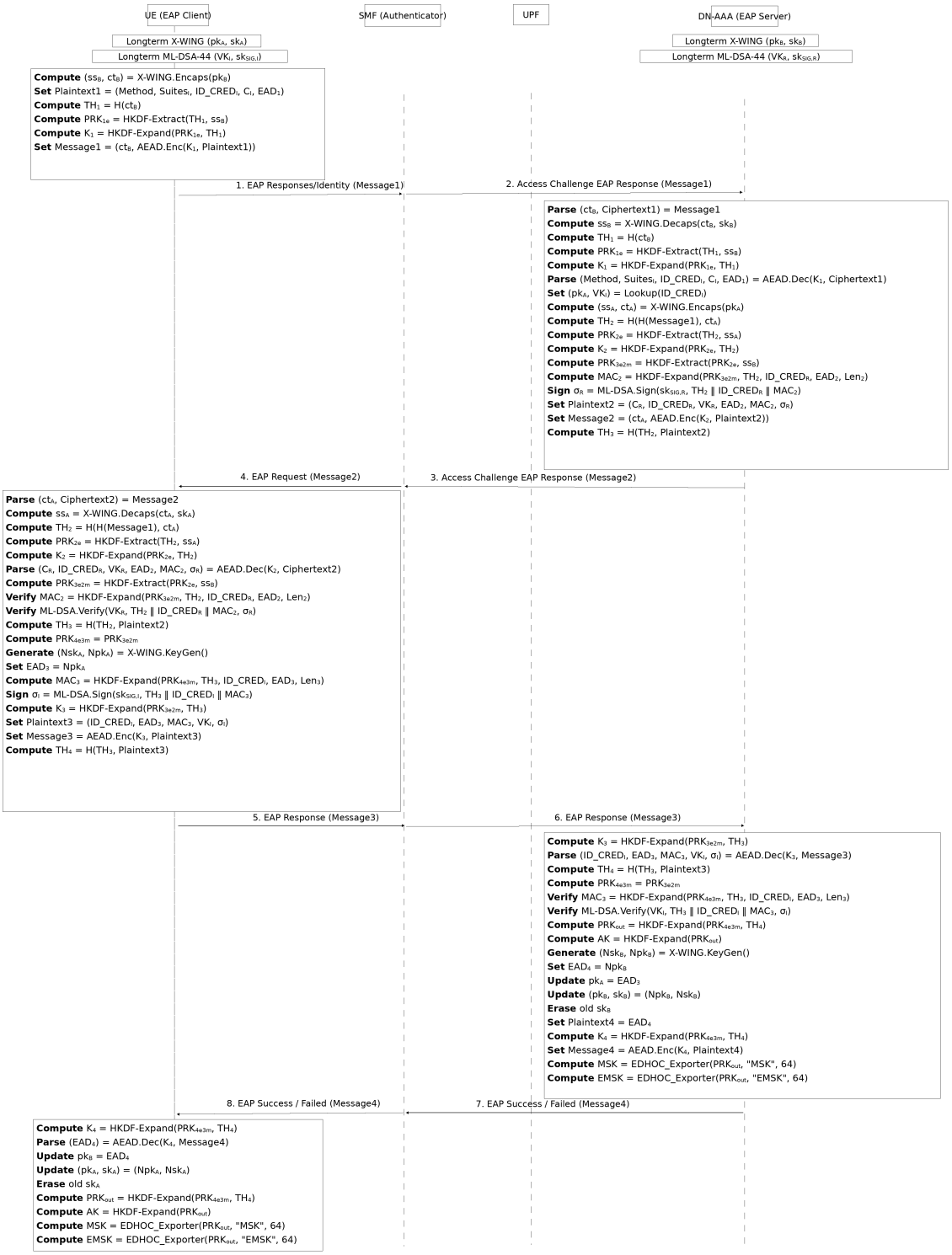


Figure 2. EAP-EDHOC hybrid PQC protocol for 5G secondary authentication.

Message 1 (Algorithm 3). The UE encapsulates a fresh secret to the DN-AAA's ~~long-term~~ current one-time key pk_B and encrypts `Plaintext1` under a transcript-derived key K_1. No ephemeral key pair is generated: forward secrecy comes from the erasure of the one-time static keys (Section 4.4). `Plaintext1` carries ID_CRED_I so that the DN-AAA can retrieve pk_A before building `Message2`. Because K_1 depends ~~on pk_B~~only on ss_B, ID_CRED_I ~~becomes retroactively recoverable if~~ is exposed if the *current* sk_B is ~~later compromised ;~~ compromised before the run; once the run completes and sk_B is erased, a later compromise no longer reveals it, which Section 5 ~~confirms this formallyand Section 7 discusses how per-session key rotation bounds it~~checks formally.

Algorithm 3 Construction of `Message1` (Initiator)

Require: ~~pk_B (long-term public key of DN-AAA)~~ pk_B (current one-time public key of the DN-AAA)
Ensure: Message1
1: ~~$(sk_X, pk_X) \leftarrow$ X-WING.KeyGen()~~ $(ss_B, ct_B) \leftarrow$ X-WING.Encaps(pk_B)
2: Plaintext1 $\leftarrow$ (Method, Suites$_I$, ID_CRED_I, C_I, EAD$_1$)
3: ~~$TH_1 \leftarrow H(pk_X, ct_B)$~~ $TH_1 \leftarrow H(ct_B)$
4: $PRK_{1e} \leftarrow$ HKDF-Extract(TH_1, ss_B)
5: $K_1 \leftarrow$ HKDF-Expand(PRK_{1e}, TH_1)
6: ~~Message1 $\leftarrow (ct_B, pk_X,$ AEAD.Enc$(K_1,$ Plaintext1$))$~~ Message1 $\leftarrow (ct_B,$ AEAD.Enc$(K_1,$ Plaintext1$))$
7: **return** Message1

Message 2 (Algorithm 4). ~~The~~ Using the UE record selected from the EAP identity, the DN-AAA decapsulates, retrieves (pk_A, VK_I) from ID_CRED_I, and encapsulates to the UE's ~~ephemeral key and to its long-term~~ one-time key pk_A~~, and~~ . The ciphertext ct_A travels in the clear part of `Message2`, and ss_A keys the encryption of `Plaintext2`, so the Responder's identity is readable only by the holder of sk_A. The DN-AAA signs $TH_2 \,\|\, ID_CRED_R \,\|\, MAC_2$ with a key independent of sk_B, giving KCI-resistant Responder authentication.

Algorithm 4 EDHOC `Message2` (DN-AAA $\rightarrow$ UE) with ML-DSA-44 authentication

Require: ~~Message1, sk_B, pk_A, $sk_{SIG,R}$, VK_R~~ Message1, per-UE records, $sk_{SIG,R}$, VK_R
Ensure: Message2
1: ~~$(ct_B, pk_X,$ Ciphertext1$) \leftarrow$ Message1~~ $(ct_B,$ Ciphertext1$) \leftarrow$ Message1
2: $ss_B \leftarrow$ X-WING.Decaps(ct_B, sk_B)
3: ~~$TH_1 \leftarrow H(pk_X, ct_B)$~~ $TH_1 \leftarrow H(ct_B)$
4: $PRK_{1e} \leftarrow$ HKDF-Extract(TH_1, ss_B)
5: $K_1 \leftarrow$ HKDF-Expand(PRK_{1e}, TH_1)
6: (Method, Suites$_I$, ID_CRED_I, C_I, EAD$_1$) $\leftarrow$ AEAD.Dec$(K_1,$ Ciphertext1$)$
7: ~~$(ss_e, ct_e) \leftarrow$ X-WING.Encaps(pk_X)~~ $(pk_A, VK_I) \leftarrow$ Lookup(ID_CRED_I)
8: ~~$TH_2 \leftarrow H(H($Message1$), ct_e)$~~ $(ss_A, ct_A) \leftarrow$ X-WING.Encaps(pk_A)
9: ~~$PRK_{2e} \leftarrow$ HKDF-Extract(TH_2, ss_e)~~ $TH_2 \leftarrow H(H($Message1$), ct_A)$
10: ~~$K_2 \leftarrow$ HKDF-Expand(PRK_{2e}, TH_2)~~ $PRK_{2e} \leftarrow$ HKDF-Extract(TH_2, ss_A)
11: ~~$(ss_A, ct_A) \leftarrow$ X-WING.Encaps(pk_A)~~ $K_2 \leftarrow$ HKDF-Expand(PRK_{2e}, TH_2)
12: $PRK_{3e2m} \leftarrow$ HKDF-Extract(PRK_{2e}, ss_B)
13: $MAC_2 \leftarrow$ HKDF-Expand$(PRK_{3e2m}, TH_2, ID_CRED_R, EAD_2, Len_2)$
14: $\sigma_R \leftarrow$ ML-DSA.Sign$(sk_{SIG,R},\ TH_2 \,\|\, ID_CRED_R \,\|\, MAC_2)$ ▷ **independent of** sk_B
15: ~~Plaintext2 $\leftarrow (ct_A, C_R, ID_CRED_R, VK_R, EAD_2, MAC_2, \sigma_R)$~~ Plaintext2 $\leftarrow (C_R, ID_CRED_R, VK_R, EAD_2, MAC_2, \sigma_R)$
16: ~~Message2 $\leftarrow (ct_e,$ AEAD.Enc$(K_2,$ Plaintext2$))$~~ Message2 $\leftarrow (ct_A,$ AEAD.Enc$(K_2,$ Plaintext2$))$
17: $TH_3 \leftarrow H(TH_2,$ Plaintext2$)$
18: **return** Message2

Message 3 (Algorithm 5). The UE verifies MAC_2 and σ_R; this signature check is an authentication *gate* that must pass before the handshake continues. The UE then authenticates itself in the mirror-image manner and announces a fresh X-Wing public key Npk_A in EAD_3.

Algorithm 5 EDHOC `Message3` (UE → DN-AAA), KCI-resistant via the ML-DSA-44 gate

Require: ~~Message2, sk_X, sk_A, ss_B, $sk_{SIG,I}$, VK_I~~ Message2, sk_A, ss_B, $sk_{SIG,I}$, VK_I
Ensure: Message3
1: ~~$(ct_e$, Ciphertext2) ← Message2~~ $(ct_A$, Ciphertext2) ← Message2
2: ~~ss_e ← X-WING.Decaps(ct_e, sk_X)~~ ss_A ← X-WING.Decaps(ct_A, sk_A)
3: ~~$TH_2 \leftarrow H(H($Message1$), ct_e)$~~ $TH_2 \leftarrow H(H($Message1$), ct_A)$
4: ~~PRK_{2e} ← HKDF-Extract(TH_2, ss_e)~~ PRK_{2e} ← HKDF-Extract(TH_2, ss_A)
5: $K_2 \leftarrow$ HKDF-Expand(PRK_{2e}, TH_2)
6: ~~$(ct_A, C_R, ID_CRED_R, VK_R, EAD_2, MAC_2, \sigma_R)$ ← AEAD.Dec$(K_2$, Ciphertext2)~~
$(C_R, ID_CRED_R, VK_R, EAD_2, MAC_2, \sigma_R) \leftarrow$ AEAD.Dec$(K_2$, Ciphertext2)
7: $PRK_{3e2m} \leftarrow$ HKDF-Extract(PRK_{2e}, ss_B)
8: **Verify** $MAC_2 \stackrel{?}{=}$ HKDF-Expand$(PRK_{3e2m}, TH_2, ID_CRED_R, EAD_2, Len_2)$ ▷ key confirmation
9: **Verify** ML-DSA.Verify$(VK_R,\ TH_2 \,\|\, ID_CRED_R \,\|\, MAC_2,\ \sigma_R) \stackrel{?}{=}$ `valid` ▷ **auth gate**
10: **if either check fails: abort**
11: $TH_3 \leftarrow H(TH_2$, Plaintext2)
12: ~~ss_A ← X-WING.Decaps(ct_A, sk_A) PRK_{4e3m} ← HKDF-Extract(PRK_{3e2m}, ss_A)~~ $PRK_{4e3m} \leftarrow PRK_{3e2m}$ ▷ ss_A already entered at PRK_{2e}
13: $(Nsk_A, Npk_A) \leftarrow$ X-WING.KeyGen(); $EAD_3 \leftarrow Npk_A$
14: $MAC_3 \leftarrow$ HKDF-Expand$(PRK_{4e3m}, TH_3, ID_CRED_I, EAD_3, Len_3)$
15: $\sigma_I \leftarrow$ ML-DSA.Sign$(sk_{SIG,I},\ TH_3 \,\|\, ID_CRED_I \,\|\, MAC_3)$ ▷ **independent of** sk_A
16: $K_3 \leftarrow$ HKDF-Expand(PRK_{3e2m}, TH_3)
17: Plaintext3 ← $(EAD_3, MAC_3, VK_I, \sigma_I)$
18: Message3 ← AEAD.Enc$(K_3$, Plaintext3$)$
19: $TH_4 \leftarrow H(TH_3$, Plaintext3$)$
20: **return** Message3

Message 4 and key export (Algorithms 6 and 7). The DN-AAA verifies the Initiator's gate, finalises the key schedule, rotates its own key pair, erases the old secret key, and returns a fresh Npk_B in EAD_4. The UE processes `Message4`, and both endpoints export identical MSK and EMSK with the EDHOC exporter. The MSK is returned to the 5G core; the EMSK is retained for key bootstrapping.

Algorithm 6 EDHOC `Message4` (DN-AAA → UE), KCI-resistant via the ML-DSA-44 gate

Require: ~~Message3, PRK_{3e2m}, TH_3, ss_A~~ Message3, PRK_{3e2m}, TH_3
Ensure: Message4
1: $K_3 \leftarrow$ HKDF-Expand(PRK_{3e2m}, TH_3)
2: $(EAD_3, MAC_3, VK_I, \sigma_I) \leftarrow$ AEAD.Dec$(K_3$, Message3$)$
3: $TH_4 \leftarrow H(TH_3$, Plaintext3$)$
4: ~~PRK_{4e3m} ← HKDF-Extract(PRK_{3e2m}, ss_A)~~ $PRK_{4e3m} \leftarrow PRK_{3e2m}$
5: **Verify** $MAC_3 \stackrel{?}{=}$ HKDF-Expand$(PRK_{4e3m}, TH_3, ID_CRED_I, EAD_3, Len_3)$ ▷ key confirmation
6: **Verify** ML-DSA.Verify$(VK_I,\ TH_3 \,\|\, ID_CRED_I \,\|\, MAC_3,\ \sigma_I) \stackrel{?}{=}$ `valid` ▷ **auth gate**
7: **if either check fails: abort**
8: $PRK_{out} \leftarrow$ HKDF-Expand(PRK_{4e3m}, TH_4); $AK \leftarrow$ HKDF-Expand(PRK_{out})
9: $(Nsk_B, Npk_B) \leftarrow$ X-WING.KeyGen(); $EAD_4 \leftarrow Npk_B$
10: $pk_A \leftarrow EAD_3$ ▷ update next key of UE
11: $(pk_B, sk_B) \leftarrow (Npk_B, Nsk_B)$; **erase** the old sk_B ▷ forward secrecy
12: Plaintext4 ← EAD_4; $K_4 \leftarrow$ HKDF-Expand(PRK_{4e3m}, TH_4)
13: Message4 ← AEAD.Enc$(K_4$, Plaintext4$)$
14: MSK ← `EDHOC_Exporter`$(PRK_{out}$, "MSK", 64$)$; EMSK ← `EDHOC_Exporter`$(PRK_{out}$, "EMSK", 64$)$
15: **return** Message4

Algorithm 7 `Message4` processing and session-key export (UE)

Require: Message4, PRK_{4e3m}, TH_4
Ensure: MSK, EMSK
1: $K_4 \leftarrow$ HKDF-Expand(PRK_{4e3m}, TH_4); $(EAD_4) \leftarrow$ AEAD.Dec$(K_4$, Message4$)$
2: $pk_B \leftarrow EAD_4$ ▷ update next key of DN-AAA
3: $(pk_A, sk_A) \leftarrow (Npk_A, Nsk_A)$; **erase** the old sk_A ▷ forward secrecy
4: $PRK_{out} \leftarrow$ HKDF-Expand(PRK_{4e3m}, TH_4); $AK \leftarrow$ HKDF-Expand(PRK_{out})
5: MSK ← `EDHOC_Exporter`$(PRK_{out}$, "MSK", 64$)$; EMSK ← `EDHOC_Exporter`$(PRK_{out}$, "EMSK", 64$)$
6: **return** (MSK, EMSK)

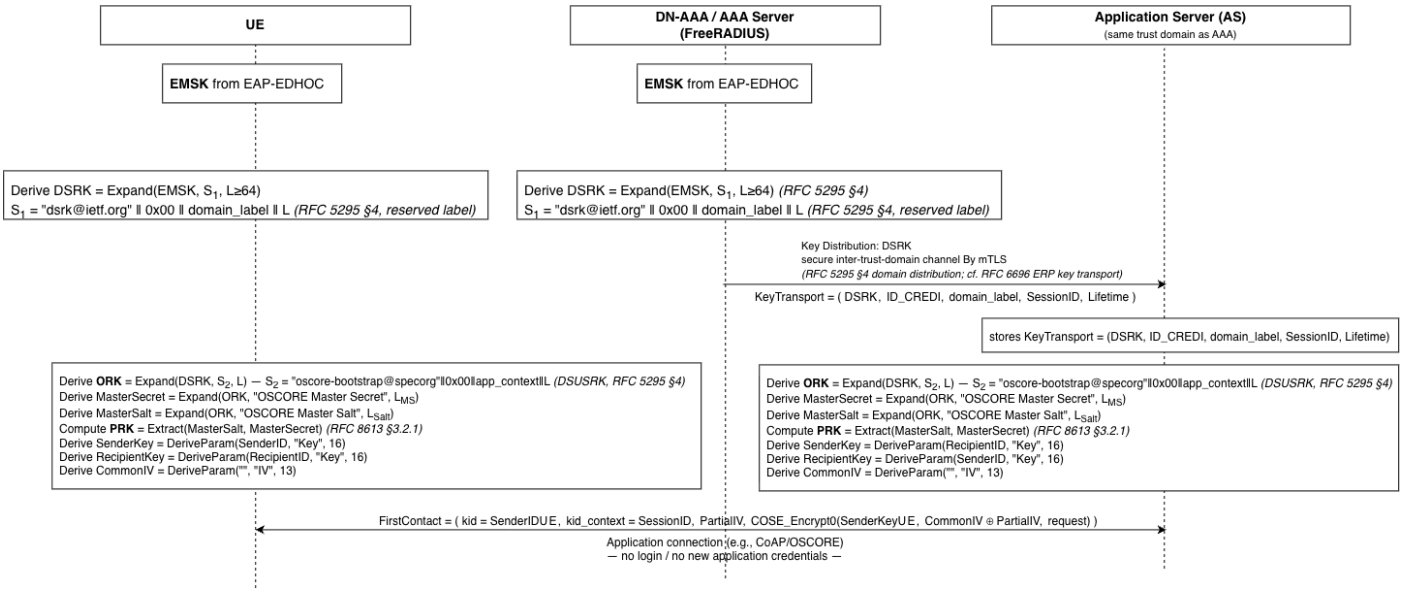


Figure 3. Application-server access through key bootstrapping from the EMSK.

4.3. EMSK-Based Key Bootstrapping

Once secondary authentication succeeds, the UE and the DN-AAA share an identical EMSK. Figure 3 shows how it bootstraps keys that let the UE access the AS directly, and Figure 4 gives the key schedule, which combines RFC 5295's root-key hierarchy [42] with the OSCORE security context of RFC 8613 [43]. Every arrow is a single Expand or Extract call with a unique label, so compromise of one derived key does not disclose keys at another level (cryptographic separation, RFC 5295 Section 3). This phase uses only symmetric-key primitives, so its cost is negligible next to the asymmetric handshake that precedes it (Section 6.4).

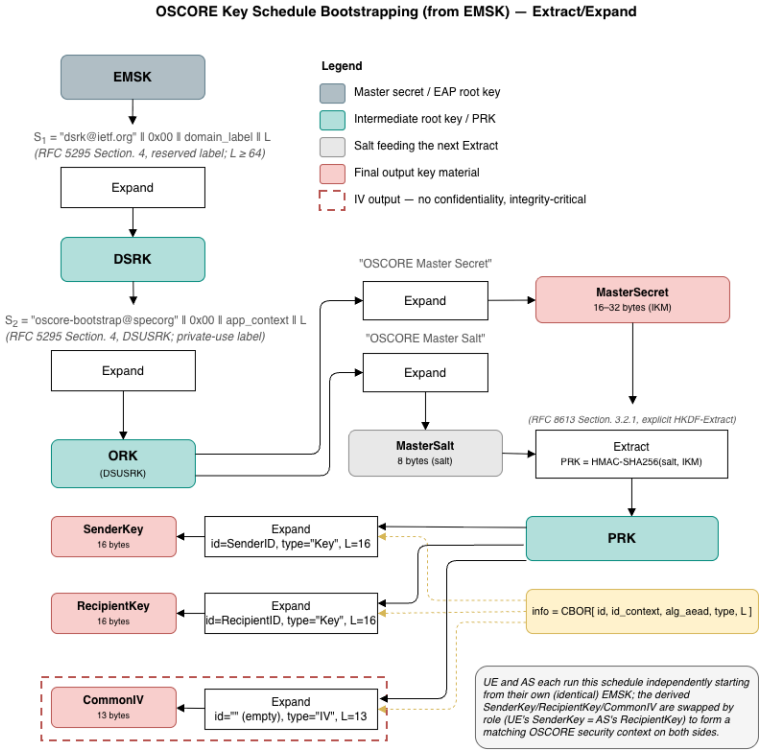


Figure 4. Key schedule for bootstrapping keys from the EMSK.

4.4. Symmetry Analysis of the Design

We call a protocol *role-symmetric* if exchanging the Initiator and the Responder maps each party's cryptographic obligations onto those of the other. The proposed protocol satisfies this property in four respects, summarised in Table 2.

(i) Symmetric implicit authentication through two encapsulations. A single KEM authenticates only the decapsulator. The protocol therefore performs ~~two static encapsulationsin opposite directions,~~ exactly two encapsulations, one in each direction: ct_B to the DN-AAA ~~(~~in `Message1` ~~)~~ and ct_A to the UE ~~(~~in `Message2`~~), and feeds both secrets into~~ . Both secrets feed the key schedule (ss_A into PRK_{2e}, ss_B into PRK_{3e2m}~~,~~), and each message is encrypted under the secret it delivers to its receiver, K_1 from ss_B and K_2 from ss_A ~~into PRK_{4e3m})~~. This reproduces, with an asymmetric primitive, the two-sided contribution that static Diffie–Hellman gives naturally.

(ii) Symmetric explicit authentication. Each party signs its own transcript hash with a detached ML-DSA-44 key (σ_R over TH_2, σ_I over TH_3), and each verification is a mandatory gate. In contrast to the mirror-image methods T1/T2, neither party relies on a static-DH MAC, so KCI resistance holds for both directions rather than for one.

(iii) Symmetric key rotation. The UE announces Npk_A in EAD_3 and the DN-AAA announces Npk_B in EAD_4; both replace their static key at the end of the run ~~. This~~ and erase the old secret key. Because every static key pair therefore serves a single session, this two-sided rotation ~~is what yields~~ yields forward secrecy (R5) as well as post-compromise security for both parties (R6), without an ephemeral key pair.

(iv) Symmetric key confirmation and output. MAC_2 and MAC_3 give mutual key confirmation, and both ends derive identical MSK/EMSK values.

The design keeps two intentional asymmetries. First, `Message1` must ~~reveal~~ carry ID_CRED_I so that the Responder can select pk_A; the Initiator's identity is therefore protected only under the Responder's ~~current long-term~~ *current* one-time key (R2). Second, the four-message flow is not size-symmetric, as `Message2` and `Message3` carry the signatures and verification keys. ~~Section 6 shows, however, that the per-party computation and the~~ The total bytes sent in each direction are ~~nearly balanced~~nevertheless balanced: 6,116 B from the UE (`Message1`+`Message3`) and 6,112 B from the DN-AAA (`Message2`+`Message4`).

Table 2. Role symmetry of the proposed protocol: corresponding obligations of the Initiator (UE) and the Responder (DN-AAA).

Obligation	Initiator (UE)	Responder (DN-AAA)
Static encapsulation received	ct_A to pk_A (in `Message2`)	ct_B to pk_B (in `Message1`)
Secret fed to key schedule	~~$ss_A \rightarrow PRK_{4e3m}$~~ $ss_A \rightarrow PRK_{2e}$	$ss_B \rightarrow PRK_{3e2m}$
Key-confirmation MAC sent	MAC_3	MAC_2
Detached signature (gate)	σ_I over TH_3, verified by DN-AAA	σ_R over TH_2, verified by UE
Static-key rotation and erasure	Npk_A in EAD_3, old sk_A erased	Npk_B in EAD_4, old sk_B erased
Session-key output	MSK, EMSK	MSK, EMSK
Intentional asymmetry	ID_CRED_I sent under the current pk_B-derived key	—

5. Formal Security Verification Using ProVerif

We analyse the protocol with ProVerif [27,28] in the Dolev–Yao model [52,55], following prior EDHOC analyses [33–35,39,56] and ISO/IEC 29128 [57]. The verification targets nine requirements:

R1 **Mutual authentication**: each party is injectively authenticated to the other (agreement on the transcript).

R2 **Identity concealment**: the persistent ~~identity~~ identities ID_CRED_I ~~is never~~ and ID_CRED_R are not learned by the adversary.

R3 **Confidentiality**: application data and session keys remain secret.

R4 **Secure key exchange**: MSK/EMSK are secret and agreed.

R5 **Forward secrecy**: past session keys remain secret ~~after~~ when the long-term ~~keys leak~~state held after the session (signing keys and current static keys) leaks.

R6 **Post-compromise security**: after a static key is compromised, security is restored once the rotated Npk_A/Npk_B is in use.

R7 **AEAD protection** (Authenticated Encryption with Associated Data): plaintexts are integrity- and confidentiality-protected.

R8 **Quantum resilience**: security holds against an adversary that can solve discrete logarithms, i.e., when X25519 and Ed25519 are broken (modelled by a quantum oracle).

R9 **KCI resistance**: compromise of a party's static KEM key does not allow impersonation of the peer.

Table 3 maps each requirement to its ProVerif queries. We model seven variants: EDHOC T0 (SIG-SIG), T1 (SIG-STAT), T2 (STAT-SIG), T3 (STAT-STAT), the proposed protocol, and the two EAP-TLS baselines of Section 6, which are modelled as certificate-authenticated TLS 1.3 handshakes over the same query set. Each variant is analysed in three settings. In the *classical* setting, an active Dolev–Yao attacker controls the network and, in phase 1, learns all long-term keys of both parties. The *quantum* setting adds an oracle, available in every phase, that returns the discrete logarithm of any X25519 element and the Ed25519 signing key of any Ed25519 public key; this models Shor's algorithm against both an active attacker and recorded traffic, while no such oracle exists for ML-KEM-768 or ML-DSA-44. In the *KCI* setting, the long-term keys of one party are known to the attacker from the start, and we check that its peer is still authenticated to it. X-Wing is modelled explicitly as ML-KEM-768 and X25519 joined by its combiner, so the quantum setting checks the hybrid property instead of assuming it. Post-compromise security is checked in a dedicated model of the proposed protocol: after a run, the old static X-Wing keys of both parties are revealed, and a second run under the keys rotated through EAD_3/EAD_4 is analysed. A property is fully secure (●) if it holds in both the classical and the quantum setting, and partial (◐) if it holds only in the classical setting. For the proposed protocol, each run uses a fresh one-time static key pair per party, and the state leaked in phase 1 is the state a device actually holds after the run: the rotated X-Wing keys and both ML-DSA-44 signing keys. A negative-control model (`edhoc_m4v2_fs_noerase.pv`) additionally leaks the old, unerased keys; there the forward-secrecy and identity queries must fail, which shows that these properties rest on key erasure and that the models are not trivially secure. All 23 models ~~were verified with~~ target ProVerif 2.05; the models, run scripts, and outputs are publicly available [32]. [Pending: run the revised models edhoc_m4v2_*.pv and confirm every status below.]

Table 3. Security requirements and verification queries.

Security Requirement	Verification Query
Mutual authentication (R1)	Q1: `event(UE_send_M1(M1))` Q2: `inj-event(UE_recv_M2(...))` ⇒ `inj-event(DN_send_M2(...))` Q3: `inj-event(DN_recv_M3(...))` ⇒ `inj-event(UE_send_M3(...))` Q4: `inj-event(UE_recv_M4(...))` ⇒ `inj-event(DN_send_M4(...))`
Identity concealment (R2)	Q5: `attacker(ID_CREDI)`; Q5b: `attacker(ID_CREDR)`
Confidentiality (R3)	Q6: `attacker(app_data_UE)` Q7: `attacker(app_data_DN)`
Secure key exchange (R4)	Q8: `attacker(MSK_test)` Q9: `attacker(EMSK_test)`
Forward secrecy (R5)	Q8 && Q9 ~~, re-evaluated~~ after `phase 1` leaks ~~all long-term keys of both parties~~ the post-run state; no-erasure control must fail
Post-compromise security (R6)	Q10: `attacker(MSK_test2)` && Q11: `attacker(EMSK_test2)` after the old static keys are revealed
AEAD protection (R7)	Q2 && Q3 && Q4 (implicit via the `AEAD_dec` rewrite)
Quantum resilience (R8)	Q8 && Q9 in the quantum setting (X25519 discrete-log and Ed25519 key-recovery oracle)
KCI resistance (R9)	Q2 && Q4 with the UE's long-term keys leaked; Q3 with the DN's long-term keys leaked

Figure 5 shows the condensed ProVerif output. The reachability query (`is false`) is a sanity check that honest runs complete. The three injective correspondences (`is true`) confirm mutual authentication with transcript agreement for both directions of the handshake (R1), which formally reflects the role symmetry of Section 4.4. The `attacker_p1` queries confirm that, even after the static keys leak, the adversary learns neither the application data (R3) nor MSK/EMSK (R4, R5), and the key-rotation model confirms that a later session under the rotated keys regains secrecy (Q10, Q11; R6). Because ~~every run also derives its keys from a fresh ephemeral X-Wing encapsulation, the specific contribution of rotation is to confine the exposure of a compromised static key to the sessions that used it. The same leak does reveal~~ the protocol has no ephemeral key pair, rotation with erasure is also what provides forward secrecy: the state leaked after a run holds only the next one-time keys, which never protected that run. As the leaked state no longer contains the erased one-time keys, neither ID_CRED_I (Q5) ~~, the intentional asymmetry discussed above~~nor ID_CRED_R (Q5b) is revealed; in the negative control, which keeps the old keys, Q5 and Q8/Q9 fail, so forward secrecy and the concealment of ID_CRED_I both rest on erasure. Because authentication rests on signatures independent of the KEM static keys, leaking sk_A/sk_B breaks neither authentication nor KCI resistance (R9); replacing the static-DH MAC of a Method 4 variant [36,37] with a SIGMA-style signature is precisely what removes this exposure.

```
Verification summary:
 • Query  not event(UE_send_M1(M1)) is false.
 • Query  inj-event(UE_recv_M2(MAC2,P2,TH2))
      ==> inj-event(DN_send_M2(MAC2,P2,TH2)) is true.
 • Query  inj-event(DN_recv_M3(MAC3,M3,TH3))
      ==> inj-event(UE_send_M3(MAC3,M3,TH3)) is true.
 • Query  inj-event(UE_recv_M4(M4,TH4))
      ==> inj-event(DN_send_M4(M4,TH4)) is true.
 • Query  not attacker_p1(ID_CREDI[]) is false.
 • Query  not attacker_p1(ID_CREDI[]) is true.
 • Query  not attacker_p1(ID_CREDR[]) is true.
 • Query  not attacker_p1(app_data_UE[]) is true.
 • Query  not attacker_p1(app_data_DN[]) is true.
 • Query  not attacker_p1(MSK_test[]) is true.
 • Query  not attacker_p1(EMSK_test[]) is true.
 • Query  not attacker(MSK_test2[]) is true.
 • Query  not attacker(EMSK_test2[]) is true.
```

Figure 5. [Expected output, to be replaced by the actual run.] ProVerif verification summary for the proposed protocol: Q1–Q9 from the base model (edhoc_m4_hybrid_base.pv) edhoc_m4v2_classical.pv and Q10–Q11 from the key-rotation model (edhoc_m4_hybrid_pcs.pvedhoc_m4v2_pcs.pv).

Table 4 compares all seven variants. The classical methods T0–T3 and EAP-TLS Classic reach R1–R5, R7, and R9 only against a classical adversary and fail R6 and R8: in their quantum-setting models, every authentication and secrecy query turns false, whereas all queries of the two hybrid variants keep their classical verdicts. EAP-TLS Hybrid PQC, which uses the same X25519MLKEM768 group and ML-DSA-44 certificates, retains its guarantees against a quantum adversary, consistent with the composition argument for hybrid combiners [18,19] and the EUF-CMA security of ML-DSA-44 [54], and with published TLS 1.3 analyses [56]. Like every other variant, however, it fails post-compromise security, because TLS 1.3 defines no static-key rotation. The T0–T3 and EAP-TLS models follow their specifications exactly, so their ○ on R6 reflects the absence of a rotation step in the standards rather than an inherent impossibility. The proposed protocol is the only variant that is fully secure on eight of the nine requirementsall nine requirements, with forward secrecy and identity concealment conditional on key erasure.

Table 4. ProVerif verification results across EDHOC authentication methods and the two EAP-TLS baselines.

Security Requirement	EAP-EDHOC Method 0 [8,10]	EAP-EDHOC Method 1 [8,10]	EAP-EDHOC Method 2 [8,10]	EAP-EDHOC Method 3 [8,10]	EAP-EDHOC Hybrid (Sec. 4)	EAP-TLS Classic [7]	EAP-TLS Hybrid PQC [7,41]
R1 Mutual authentication	◐	◐	◐	◐	●	◐	●
R2 Identity concealment	◐	◐	◐	◐	●†	◐	●
R3 Confidentiality	◐	◐	◐	◐	●	◐	●
R4 Secure key exchange	◐	◐	◐	◐	●	◐	●
R5 Forward secrecy	◐	◐	◐	◐	●	◐	●
R6 Post-compromise security	○	○	○	○	●	○	○
R7 AEAD protection	◐	◐	◐	◐	●	◐	●
R8 Quantum resilience	○	○	○	○	●	○	●
R9 KCI resistance	◐	◐	◐	◐	●	◐	●

○ not secure; ◐ partial (secure only against a classical adversary); ● fully secure (classical *and* quantum adversary). Every entry is a ProVerif result, except the ○ marks on R6 for the six baselines, whose specifications define no static-key rotation. †ID_CRED_I is sent in `Message1` under a key derived from the DN-AAA's current one-time key. It stays secret after that key is erased, but is exposed if the current sk_B is compromised before the run. [Statuses of the proposed protocol to be confirmed by the runs of `edhoc_m4v2_*.pv`.]

6. Performance Evaluation

6.1. Testbed and Cryptographic Cost

Experiments ran on a Windows 11 Pro host (Intel Core i9-14900K, 64 GB DDR5) in a VirtualBox VM with Ubuntu 24.04 LTS (16 GB RAM, 12 vCPUs). The testbed uses Open5GS [29], UERANSIM [30], and FreeRADIUS [31], all co-located in the VM and communicating over loopback, so the measured network overhead reflects protocol-stack, scheduling, and RADIUS processing rather than RAN/WAN propagation and should be read as a lower bound. We modified Open5GS 2.7.2, UERANSIM 3.2.7, and FreeRADIUS 3.2.6. The EDHOC methods (0–4) use libsodium [58] for X25519, Ed25519, SHA-256, and HKDF, mbedTLS [59] for AES-128-GCM (both from the Ubuntu 24.04 LTS repositories), and the PQClean [60] clean reference implementations of ML-KEM-768 and ML-DSA-44. The EAP-TLS baselines use OpenSSL: EAP-TLS Classic runs on the system OpenSSL 3.0.13, while EAP-TLS Hybrid PQC runs on a separately built OpenSSL 3.2.6 with liboqs 0.16 [61] and oqs-provider 0.12 [62], because ML-DSA-44 certificates in TLS 1.3 require OpenSSL 3.2 or later; each Hybrid PQC handshake is checked at run time to have negotiated the X25519MLKEM768 group and an ML-DSA-44 certificate. The source code, configuration, and benchmark scripts are publicly available [32]. EAP-EDHOC runs over real EAP-over-RADIUS (RFC 2865/3579) [21,22] to a FreeRADIUS `rlm_eap_edhoc` responder along the path UE → AMF → SMF → UPF → FreeRADIUS. All five EDHOC methods are implemented: methods 0–3 use three messages and method 4 (the proposal) four messages with EAP fragmentation. EAP-TLS Classic and EAP-TLS Hybrid PQC are benchmarked end to end on the same testbed.

During the preparation of this manuscript, the authors used Claude (Anthropic; model claude-opus-5-5) to convert the manuscript to the journal template and to condense and restructure the text; the tool was not used to generate data, results, or analyses. The authors reviewed and edited the output and take full responsibility for the content of this publication.

Table 5 lists primitive costs over 3000 iterations. The most expensive operation, ML-DSA-44 signing, averages 0.238 ms, and all ML-KEM-768 operations stay below 0.05 ms. Per handshake, the proposed method costs ~~1.259~~1.039 ms, split ~~almost~~ exactly evenly between the initiator ~~(0.651 ms)~~ and the responder (~~0.608 ms), a direct consequence of the role symmetry of Section~~ ~~4.4~~0.520 ms each): apart from generating its next key pair, each party performs exactly one X-Wing encapsulation and one decapsulation, one signature and one verification, so the two parties run mirror-image operation sets. The classical methods 0–3 cost 0.148–0.180 ms, and EAP-TLS Classic and Hybrid PQC 0.180 ms and 0.820 ms. PQC thus multiplies computation by about ~~7~~6–~~8~~×7×, yet the absolute cost stays ~~under 1.5~~near 1 ms. Figures 6 and 7 break down computation and communication for all seven methods.

Table 5. Mean latency of cryptographic primitives (3000 iterations).

Operation	Mean (ms)	StdDev (ms)	95% CI (±ms)
Keygen_X25519	0.024004	0.028271	0.001012
ScalarMult_X25519	0.024889	0.016870	0.000604
Keygen_Ed25519	0.010882	0.012333	0.000441
Signature_Ed25519	0.011226	0.016348	0.000585
Verify_Ed25519	0.029693	0.021514	0.000770
SHA256	0.000263	0.000003	0.000000
HKDF_SHA256	0.001448	0.005969	0.000214
AES128GCM	0.000255	0.001266	0.000045
Keygen_MLKEM768	0.034582	0.019357	0.000693
Encaps_MLKEM768	0.039528	0.047955	0.001716
Decaps_MLKEM768	0.047708	0.034353	0.001229
Keygen_MLDSA44	0.057264	0.039540	0.001415
Signature_MLDSA44	0.237660	0.203541	0.007284
Verify_MLDSA44	0.062323	0.041196	0.001474

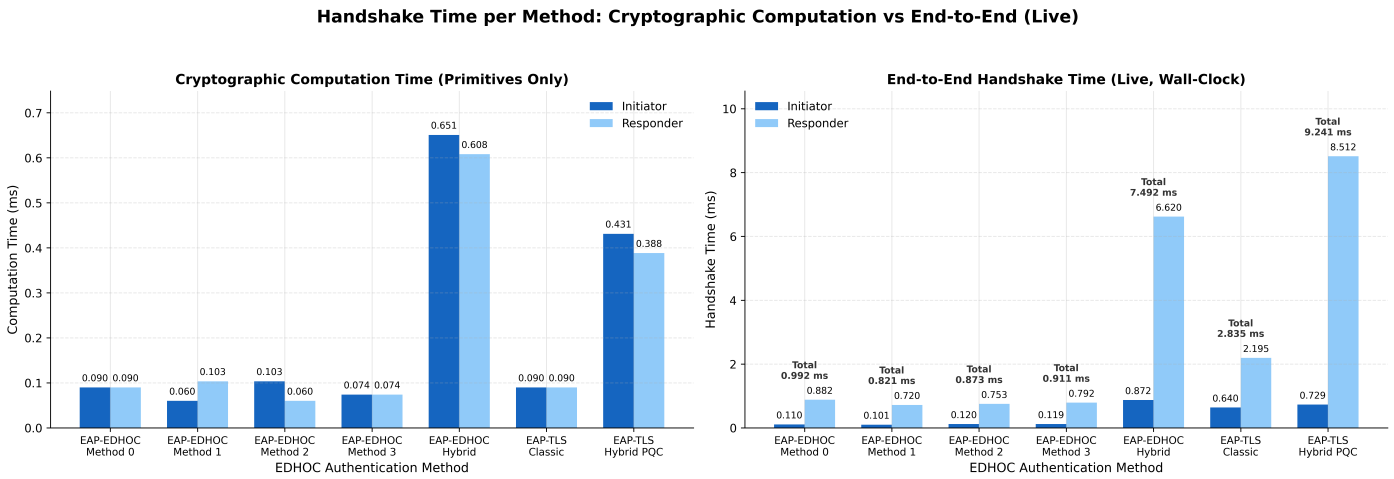


Figure 6. Per-method handshake computation cost (**left**) and end-to-end handshake time (**right**).

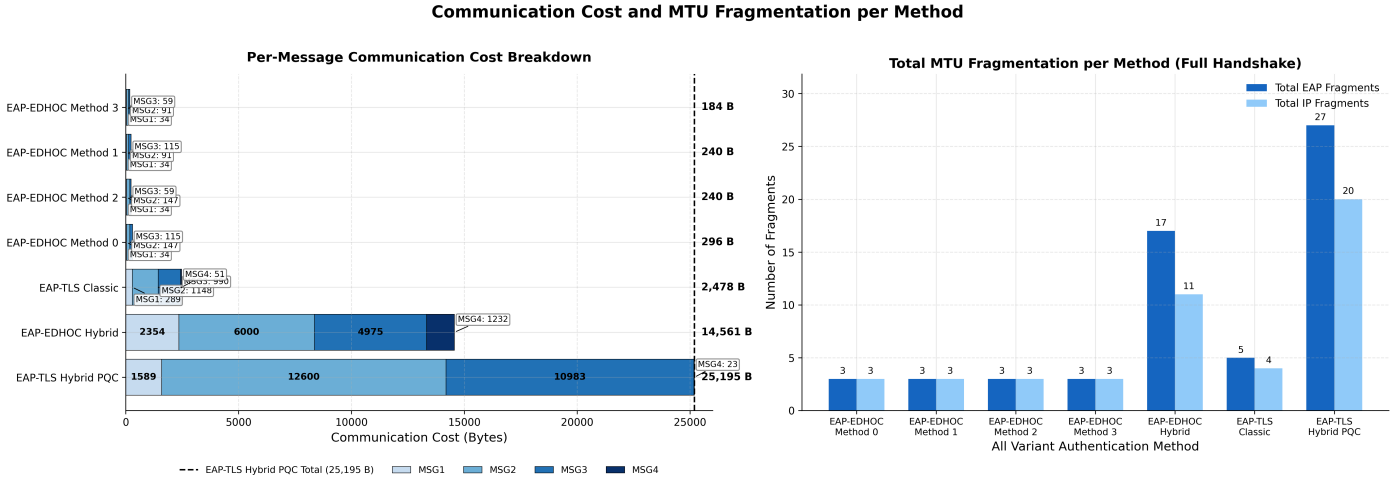


Figure 7. Per-method communication (message size) breakdown and MTU fragmentation.

6.2. End-to-End Latency, Message Size, and Framing

Over the live EAP-over-RADIUS path (100 iterations per method, no loss), the proposed method completes in ~~7.492 ms on average(median 5.410 ms, p95 14.226 ms) with a 100% success rate~~[re-measure] on average, against 0.821–0.992 ms for methods 0–3, 2.835 ms for EAP-TLS Classic, and 9.241 ms for EAP-TLS Hybrid PQC. The gap is driven by fragmentation, not computation: the proposal needs ~~17~~ [re-measure] RADIUS round-trips versus 3 for methods 0–3, and EAP-TLS Classic and Hybrid PQC need 6 and 28.

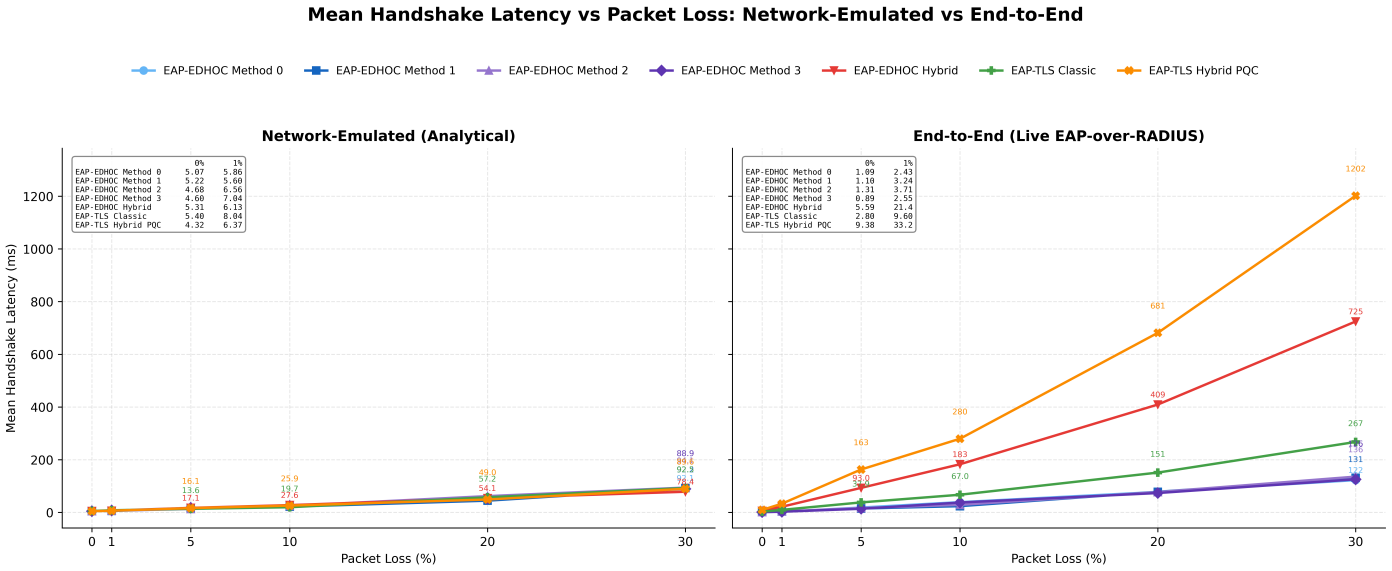
The message sizes explain this. Classical messages fit in one EAP fragment (34–147 bytes). In the proposed method, each X-Wing element adds 1.1–1.2 KB and each ML-DSA-44 signature about 2.4 KB~~, so~~ . `Message1` (~~≈ 2.35 KB) ,~~ 1,141 B: ct_R and the encrypted `Plaintext1`) now fits within the 1500-byte MTU, whereas `Message2` (~~≈ 6.0 KB) ,~~ 4,880 B) and `Message3` (~~≈ 5.0 KB) exceed the 1500-byte MTU~~ 4,975 B) exceed it and are split into EAP fragments (RFC 3579, 253-byte attributes) [22] and IP fragments~~, while~~ ; `Message4` (~~≈ 1.2 KB~~1,232 B) fits in one. ~~Although individual messages differ widely in size, the~~ The handshake totals 12,228 B. The load per direction is ~~nearly~~ balanced: the UE sends ~~about 7.35 KB~~ 6,116 B (`Message1+Message3`) and the DN-AAA ~~about 7.2 KB~~ 6,112 B (`Message2+Message4`). EAP-TLS Classic totals 2,478 B in 5 EAP fragments and EAP-TLS Hybrid PQC 25,195 B in 27, the most of any method. Cross-implementation checks against pyca/cryptography over OpenSSL passed all 11 tests, with sizes matching the standards

(ML-KEM-768 public key 1184 B, ciphertext 1088 B; ML-DSA-44 key 1312 B, signature 2420 B).

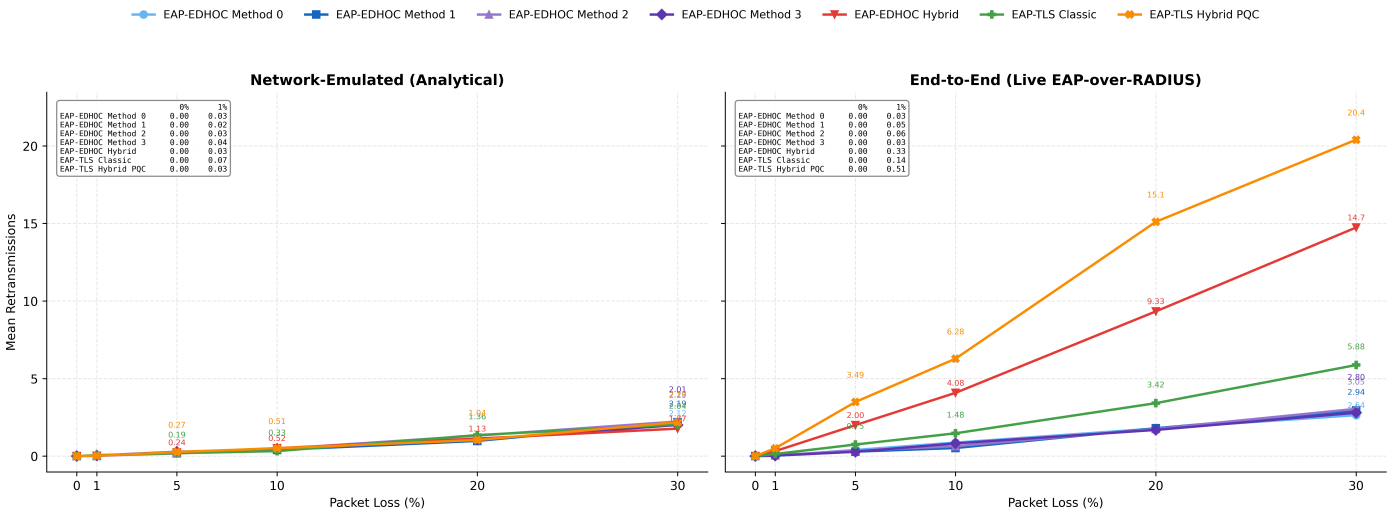
The two panels of Figure 6 rank the methods differently. In computation, the proposal is the most expensive (~~1.259~~1.039 ms vs. 0.820 ms for EAP-TLS Hybrid PQC) because ~~its four messages run~~ it runs more X25519 and ML-KEM-768 operations; both perform the same signatures. End to end, ~~the order reverses (9.241 ms vs. 7.492 ms) because~~ EAP-TLS carries ML-DSA-44 inside an X.509 chain in a TLS record, whereas EAP-EDHOC carries a compact detached CBOR/COSE value: 25,195 B and 28 round-trips against ~~14,561~~12,228 B and ~~17~~[re-measure] round-trips. The same crossover appears classically: EAP-TLS Classic and Method 0 have identical compute cost (0.180 ms, both Ed25519 and one X25519 exchange; we used an Ed25519 certificate instead of RSA-2048 to isolate this effect), yet EAP-TLS Classic is 2.9× slower end to end (2.835 vs. 0.992 ms) due to TLS/X.509 framing and 6 round-trips. EDHOC's compact encoding, not its primitives, keeps it below EAP-TLS at every security level, and the gap widens with PQC.

6.3. Behaviour under Packet Loss

Figures 8 and 9 report behaviour under emulated application-level loss with EAP-style retransmission (RTO 40 ms, up to 6 retransmissions per exchange, 120 trials per cell) for all seven methods. Methods 0–3 keep a 100% success rate up to 20% loss, while the larger-payload methods degrade earlier: ~~the proposal and~~ EAP-TLS Classic ~~reach~~ reaches 99.2% and EAP-TLS Hybrid PQC 96.7% at 20% loss~~. At~~ , and 95.8% and 73.3% at 30% loss, while the classical methods remain at 95.8–97.5%~~, EAP-TLS Classic at 95.8%, the proposaldrops to 87.5%, and EAP-TLS Hybrid PQC to 73.3%~~. For the proposal: [re-measure success rate, latency, and retransmissions at 20% and 30% loss]. Each extra fragment is an independent chance of loss, so latency and retransmissions grow with fragment count~~: the proposal reaches 409.3 ms and 9.3 retransmissions at 20% loss and 724.6 ms and 14.7 at 30%, while EAP-TLS Hybrid PQC reaches 1201.8 ms and 20.4 at 30%~~. Fragmentation, not computation, is the main scalability concern.

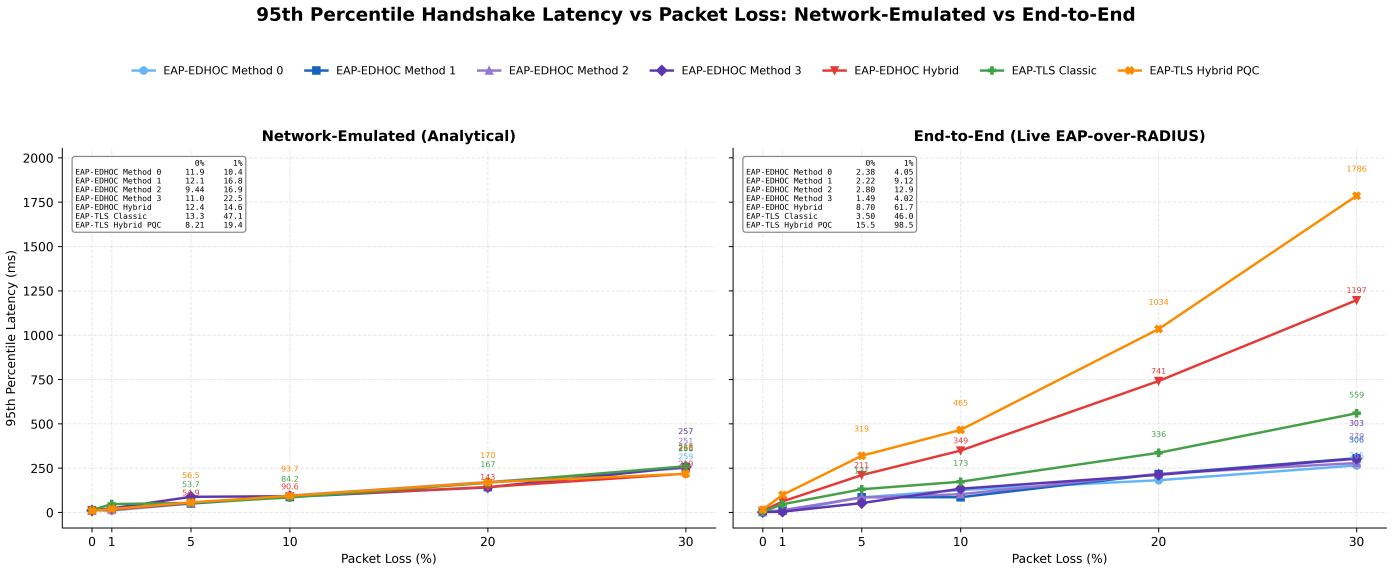


(**a**) Mean end-to-end latency versus packet loss.

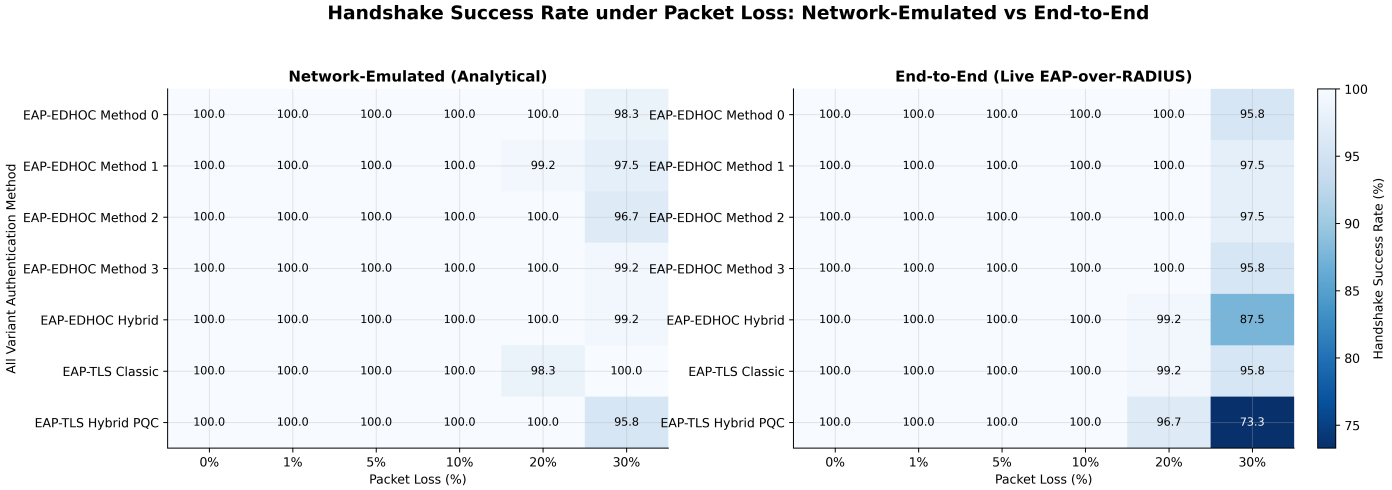


(**b**) Mean number of retransmissions versus packet loss.

Figure 8. Mean behaviour under packet loss: (**a**) end-to-end latency; (**b**) retransmissions.



(**a**) 95th-percentile end-to-end latency versus packet loss.

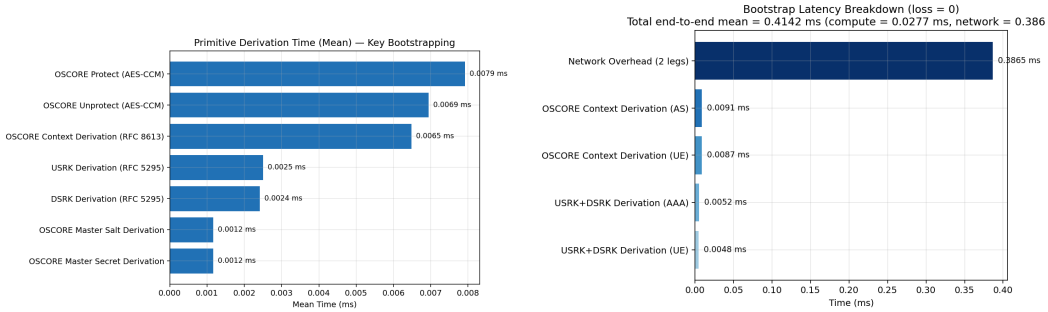


(**b**) Handshake success rate across methods and loss levels.

Figure 9. Tail latency and reliability under packet loss: (**a**) 95th-percentile latency; (**b**) success-rate heatmap.

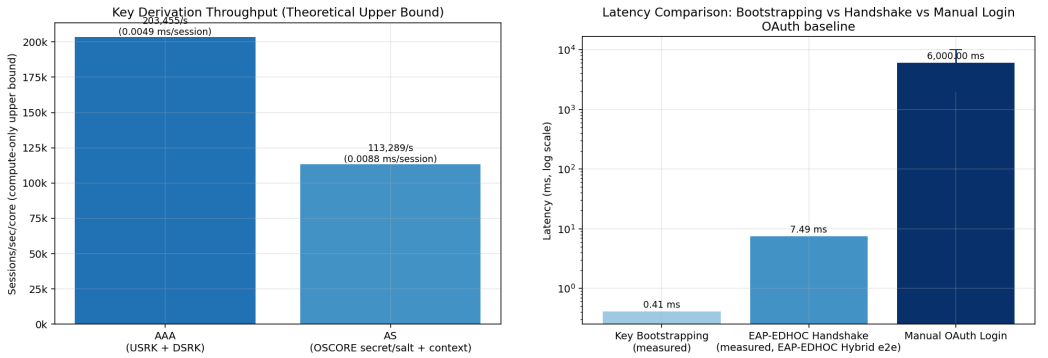
6.4. EMSK Key-Bootstrapping Cost

Every primitive in the bootstrapping chain is inexpensive (Figure 10a), ranging from 0.0012 ms (OSCORE Master Secret/Salt derivation) to 0.0079 ms (OSCORE Protect, AES-CCM). Of the loss-free end-to-end bootstrap latency of 0.4142 ms (Figure 10b), only 0.0277 ms is spent on the four derivation steps (USRK+DSRK on the UE and AAA sides, OSCORE context on the UE and AS sides); the remaining 0.3865 ms is network overhead on the two new legs, AAA→AS (DSRK transport) and UE→AS (first OSCORE contact). The single-core throughput bound (Figure 11a) is 203,455 sessions/s (4.915 µs/session) on the AAA side and 113,289 sessions/s (8.827 µs/session) on the AS side, so bootstrapping does not shift the bottleneck away from the AAA server. At 0.41 ms, bootstrapping is ~~about 18×~~ far cheaper than the ~~7.49 ms handshake~~ handshake ([re-measure]) and far below the 2–10 s typically reported for a manual OAuth 2.0 authorization-code login, at least 4,800× faster than the 2 s lower bound (Figure 11b).



(**a**) (**b**)

Figure 10. EMSK bootstrapping cost: (**a**) mean latency of each primitive; (**b**) compute and network breakdown of the end-to-end bootstrap.



(**a**) (**b**)

Figure 11. EMSK bootstrapping scalability: (**a**) single-core throughput; (**b**) comparison with the handshake and a manual login.

Two implementation notes apply. The label `"dsrk@ietf.org"` is the reserved label of RFC 5295 Section 4, whereas `"oscore-bootstrap@<specorg>"` for the *ORK* step is a proposed private-use label (RFC 5295 Section 8), not an IANA registration. Cryptographic separation holds because every level uses distinct labels on the same Expand/Extract function: compromising *ORK* in one domain does not reveal another domain's *ORK*, and *DSRK* does not reveal *EMSK* (assuming HMAC-SHA-256 is a PRF); *id_context* separates multiple OSCORE sessions to the same AS without a new *ORK*. RFC 5295 defines no *DSRK* transport; RFC 5749 [63] requires integrity, replay protection, and key wrapping, which we meet with mutual TLS 1.3 between the DN-AAA and the AS, as both sit in one trust domain without an AAA proxy. The chain is purely symmetric and independent of the ML-DSA keys, so its security is inherited from the EMSK.

7. Discussion

Symmetric guarantees, asymmetric costs. The results show two distinct symmetry structures. In the *security* dimension the protocol is role-symmetric: both parties are authenticated by independent signatures, both are KCI-resistant, both rotate ~~their~~ and erase their one-time keys, and ~~the computation is balanced (0.651 vs. 0.608 ms), as is~~ both the computation (0.520 ms per party) and the traffic per direction (~~≈ 7.35~~ 6,116 vs. ~~≈ 7.2 KB)~~ 6,112 B) are balanced. In the *cost* dimension it is strongly asymmetric: computation is ~~1.259 ms of a 7.492 ms~~ a small part of the handshake, and the rest is fragmentation over ~~17~~ RADIUS round-trips ([re-measure]). This decomposition is actionable. Faster PQC primitives would barely change end-to-end latency, whereas the transport cost can be cut

today by raising the EAP fragment size or the RADIUS/MTU budget on the SMF–DN-AAA path, without touching the verified cryptography.

Design choices against the closest prior art. The KEM-based, signature-free method of Fraile et al. [24,25] would save the ML-DSA-44 sign/verify cost (≈ 0.238 ms and 0.062 ms), which is ~~immaterial against a 7.492 ms handshake~~ small against the end-to-end handshake time, while it is precisely the detached signature that gives formally verified KCI resistance independent of the KEM static key (R9). It would also break the explicit-authentication symmetry of Section 4.4, since a KEM authenticates only the decapsulator. Shin et al.'s Hybrid-EDHOC [26] omits Message 4 to save payload, yet Message 4 is the smallest message (1,232 B, one EAP fragment; Figure 7) and carries the Responder's half of the symmetric key rotation that yields post-compromise security (R6). Omitting it would barely reduce fragmentation, since `Message2` and `Message3` dominate the ~~14,561~~12,228 B, but would lose a property that T0–T3 lack entirely.

Mitigating fragmentation. Three mitigations require no protocol change: (i) raising the EAP fragment size and RADIUS attribute budget beyond the RFC 3579 253-byte default where the operator controls both endpoints; (ii) raising the path MTU on the SMF–UPF–DN-AAA segment, which never crosses the public Internet; and (iii) adopting more compact PQC parameter sets or signatures as they mature, which would shrink `Message2` and `Message3`. None of these affect the authentication logic verified in Section 5.

Threats to validity. (1) All components share one VM over loopback, so the network figures are a lower bound; a smaller path MTU or higher RTT in a distributed deployment would make fragmentation more pronounced. (2) Loss is emulated at the application layer rather than with `netem`, so the relative sensitivity of each method generalises better than the absolute milliseconds. (3) 100 iterations and 120 trials per cell separate methods that differ by an order of magnitude but are modest for tail estimates among methods 0–3, whose differences are tens of microseconds (Table 5). (4) The formal analysis is symbolic, and the quantum adversary is captured by a discrete-log oracle rather than a computational model; a computational proof is future work. (5) Trust in the AAA→AS DSRK channel is a co-location/mTLS assumption analogous to ERP [64], outside the ProVerif model. (6) ~~The rotation-bounded exposure of ID_CRED_I (R2) is not formally modelled; an epoch-based model with per-session key state is future work~~Forward secrecy now rests on the rotation completing and on the erasure of the old secret keys. If `Message4` is lost, the UE keeps its old key while the DN-AAA has rotated; keeping the old key for recovery delays forward secrecy for that run, whereas discarding it requires re-provisioning. The models abstract a completed rotation and do not cover this recovery path. Likewise, if the *current* one-time keys of both parties leak before a run, a passive adversary can read that run, since no ephemeral secret remains; security returns from the next run because the rotated secret keys never leave the devices. (7) Re-authentication and handover, including the fate of DSRK/OSCORE state at the AS, are not evaluated.

8. Conclusions

We proposed and implemented a role-symmetric hybrid post-quantum EAP-EDHOC protocol for 5G secondary authentication. X-Wing (X25519 + ML-KEM-768) provides key agreement through exactly two opposite-direction encapsulations to one-time static keys, detached ML-DSA-44 signatures authenticate each party independently of its KEM ~~static~~ key, and a two-sided ~~key rotation provides~~ rotation with key erasure provides both forward secrecy and post-compromise security without an ephemeral key pair. ProVerif verification across seven methods shows that only the proposal combines quantum resilience (R8), KCI resistance, forward secrecy, and post-compromise security; identity concealment (R2) holds except under compromise of the Responder's current ~~long-term~~

~~key , an exposure bounded by per-session rotation~~one-time key before the run. The exported EMSK bootstraps OSCORE keys following RFC 5295 and RFC 8613, so the UE reaches an application server without further login.

On Open5GS, UERANSIM, and FreeRADIUS, the handshake ~~costs under 1.5 ms of balanced computation~~ exchanges 12,228 B, split evenly between the two directions, and completes in ~~7.492 ms on averagewith a 100% success rate without loss~~[re-measure] on average. Its cost is asymmetric, concentrated in fragmentation ~~(17 RADIUS round-trips versus 3 classically)~~rather than computation, which dominates under high packet loss. Future work will reduce this overhead through compact PQC parameter sets, EAP fragment-size and MTU tuning, evaluation under mobility and handover with EMSK re-bootstrapping, a computational security proof, and IANA registration of the OSCORE-bootstrap label. Quantum-resistant secondary authentication is thus deployable on existing 5G infrastructure, with security that is symmetric between the parties and a cost that lies in transport rather than cryptography.

Author Contributions: Conceptualization, A.P.S.I., S.S. and I.Y.; methodology, A.P.S.I.; software, A.P.S.I., I.W.A.J.P. and H.K.; formal analysis, A.P.S.I. and Y.K.; investigation, A.P.S.I., Y.K., I.W.A.J.P. and H.K.; data curation, A.P.S.I., Y.K., I.W.A.J.P. and H.K.; writing original draft preparation, A.P.S.I. and S.S.; writing review and editing, A.P.S.I. and S.S.; supervision, I.Y. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korean government(MSIT) (No. RS-2024-00398312, Development of Quantum Security Based Device Identification and Test Verification Technologies for 5G Non-Public Network)

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The source code of the modified Open5GS, UERANSIM, and FreeRADIUS components, the EAP-EDHOC implementation, the ProVerif models, the benchmark scripts, and the raw measurement results supporting this study are openly available in the EdhocSecondaryAuth repository at https://github.com/adipanca/EdhocSecondaryAuth [32].

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

3GPP	3rd Generation Partnership Project
5G-AKA	5G Authentication and Key Agreement
AAA	Authentication, Authorization, and Accounting
AEAD	Authenticated Encryption with Associated Data
AMF	Access and Mobility Management Function
AS	Application Server
AUSF	Authentication Server Function
CBOR	Concise Binary Object Representation
COSE	CBOR Object Signing and Encryption
DN	Data Network
DSRK	Domain-Specific Root Key
EAP	Extensible Authentication Protocol
EDHOC	Ephemeral Diffie–Hellman Over COSE
EMSK	Extended Master Session Key
HKDF	HMAC-based Key Derivation Function
KCI	Key-Compromise Impersonation
KEM	Key Encapsulation Mechanism
MEC	Multi-access Edge Computing
ML-DSA	Module-Lattice-Based Digital Signature Algorithm
ML-KEM	Module-Lattice-Based Key-Encapsulation Mechanism
MSK	Master Session Key
MTU	Maximum Transmission Unit
OSCORE	Object Security for Constrained RESTful Environments
PQC	Post-Quantum Cryptography
RADIUS	Remote Authentication Dial-In User Service
SMF	Session Management Function
UE	User Equipment
UPF	User Plane Function
USRK	Usage-Specific Root Key

References

1. 3GPP TS 33.501. Security architecture and procedures for 5G System. https://www.3gpp.org/DynaReport/33501.htm, 2024. Release 18. Accessed on 5 March 2026.
2. 3GPP TS 23.501. System architecture for the 5G System (5GS). https://www.3gpp.org/DynaReport/23501.htm, 2024. Release 18. Accessed on 5 March 2026.
3. Edris, E.K.K.; Aiash, M.; Loo, J.K.K. Formal Verification and Analysis of Primary Authentication based on 5G-AKA Protocol. In Proceedings of the 3rd International Symposium on 5G Emerging Technologies (5GET), in conjunction with the 4th International Conference on Fog and Mobile Edge Computing (FMEC). IEEE, 2020, pp. 256–261.
4. Borgaonkar, R.; Hirschi, L.; Park, S.; Shaik, A. New Privacy Threat on 3G, 4G, and Upcoming 5G AKA Protocols. *Proceedings on Privacy Enhancing Technologies* **2019**, *2019*, 108–127.
5. IETF RFC 3748. Extensible Authentication Protocol (EAP). https://www.rfc-editor.org/rfc/rfc3748.html, 2004. Accessed on 5 March 2026.
6. Gong, S.; EL Azzaoui, A.; Cha, J.; Park, J.H. Secure Secondary Authentication Framework for Efficient Mutual Authentication on a 5G Data Network. *Applied Sciences* **2020**, *10*, 727.
7. IETF RFC 5216. The EAP-TLS Authentication Protocol. https://www.rfc-editor.org/rfc/rfc5216.html, 2008. Accessed on 5 March 2026.
8. IETF RFC 9528. Ephemeral Diffie-Hellman Over COSE (EDHOC). https://www.rfc-editor.org/rfc/rfc9528.html, 2024. Accessed on 1 February 2026.
9. IETF RFC 9529. Traces of Ephemeral Diffie-Hellman Over COSE (EDHOC). https://www.rfc-editor.org/rfc/rfc9529.html, 2024. Accessed on 2 February 2026.
10. IETF Internet-Draft. Extensible Authentication Protocol (EAP) using EDHOC (EAP-EDHOC). https://datatracker.ietf.org/doc/draft-ietf-emu-eap-edhoc/, 2025. Accessed on 5 March 2026.
11. Fedrecheski, G.; Vučinić, M.; Watteyne, T. Performance Comparison of EDHOC and DTLS 1.3 in Internet-of-Things Environments. In Proceedings of the Wireless Communications and Networking Conference (WCNC) 2024. IEEE, 2024.

12. Pérez, E.L.; Selander, G.; Mattsson, J.P.; Watteyne, T.; Vučinić, M. EDHOC is a New Security Handshake Standard: An Overview of Security Analysis. *Computer* **2024**, *57*, 101–110.
13. Shor, P. Algorithms for quantum computation: discrete logarithms and factoring. In Proceedings of the Proceedings 35th Annual Symposium on Foundations of Computer Science. IEEE, 1994, pp. 124–134.
14. NIST. Post-Quantum Cryptography. https://csrc.nist.gov/projects/post-quantum-cryptography. Accessed on 26 March 2026.
15. Migration to Post-Quantum Cryptography. NIST Crypto Agility Guide. Accessed on 14 February 2026.
16. ANSSI views on the Post-Quantum Cryptography transition. https://cyber.gouv.fr/en/publications/anssi-views-post-quantum-cryptography-transition. Accessed on 14 February 2026.
17. IETF Internet-Draft. Terminology for Post-Quantum Traditional Hybrid Schemes. https://www.ietf.org/archive/id/draft-ietf-pquip-pqt-hybrid-terminology-04.html, 2024. Accessed on 10 February 2026.
18. Bindel, N.; Brendel, J.; Fischlin, M.; Goncalves, B.; Stebila, D. Hybrid Key Encapsulation Mechanisms and Authenticated Key Exchange. In Proceedings of the PQCrypto 2019. Springer, 2019, pp. 206–226.
19. Barbosa, M.; Connolly, D.; Duarte, J.D.; Kaiser, A.; Schwabe, P.; Varner, K.; Westerbaan, B. X-Wing: The Hybrid KEM You've Been Looking For. *IACR Communications in Cryptology* **2024**, *1*. https://doi.org/10.62056/a3qj89n4e.
20. IETF Internet-Draft. X-Wing: General-Purpose Hybrid Post-Quantum KEM. https://datatracker.ietf.org/doc/draft-connolly-cfrg-xwing-kem/, 2025. Accessed on 5 March 2026.
21. IETF RFC 2865. Remote Authentication Dial In User Service (RADIUS). https://www.rfc-editor.org/rfc/rfc2865.html, 2000. Accessed on 5 March 2026.
22. IETF RFC 3579. RADIUS (Remote Authentication Dial In User Service) Support For Extensible Authentication Protocol (EAP). https://www.rfc-editor.org/rfc/rfc3579.html, 2003. Accessed on 5 March 2026.
23. Fraile, L.P.; Tasopoulos, G.; Koulamas, C.; Zhao, R.K.; Sultan, N.H.; Regazzoni, F.; Fournaris, A.P. Enabling Quantum-Resistant EDHOC: Design and Performance Evaluation. *IEEE Access* **2025**, *13*, 75861–75884.
24. Fraile, L.P.; Koulamas, C.; Fournaris, A.P. Reinventing EDHOC for the Post-Quantum Era. *IEEE Access* **2025**, *13*, 196622–196639. https://doi.org/10.1109/ACCESS.2025.3633843.
25. IETF Internet-Draft. KEM-based Authentication for EDHOC. https://www.ietf.org/archive/id/draft-pocero-authkem-edhoc-02.html, 2026. Accessed on 26 March 2026.
26. Shin, S.; Iskandar, A.P.S.; Ko, Y.; You, I. A Hybrid EDHOC Protocol. In Proceedings of the The 9th International Conference on Mobile Internet Security (MobiSec 2025), 2025.
27. Blanchet, B. An Efficient Cryptographic Protocol Verifier Based on Prolog Rules. In Proceedings of the CSFW, 2001.
28. Blanchet, B. Automated Verification of Correspondences for Security Protocols. *Journal of Computer Security* **2016**.
29. Lee, S. Open5GS: Open Source implementation of 5G Core and EPC. https://open5gs.org/, 2024. Accessed on 5 March 2026.
30. Güngör, A. UERANSIM: Open source 5G UE and RAN (gNodeB) simulator. https://github.com/aligungr/UERANSIM, 2024. Accessed on 5 March 2026.
31. The FreeRADIUS Server Project. FreeRADIUS: The world's most popular RADIUS Server. https://www.freeradius.org/, 2024. Accessed on 5 March 2026.
32. Iskandar, A.P.S. EdhocSecondaryAuth: Hybrid Post-Quantum EAP-EDHOC Secondary Authentication over Open5GS, UERANSIM, and FreeRADIUS (Source Code, ProVerif Models, and Benchmark Data). https://github.com/adipanca/EdhocSecondaryAuth, 2026. Accessed on 13 October 2026.
33. Jacomme, C.; Klein, E.; Kremer, S.; Racouchot, M. A comprehensive, formal and automated analysis of the EDHOC protocol. In Proceedings of the USENIX Security Symposium 2023. USENIX Association, 2023, pp. 5881–5898.
34. Norrman, K.; Sundararajan, V.; Bruni, A. Formal Analysis of EDHOC Key Establishment for Constrained IoT Devices. In Proceedings of the SECRYPT 2021. SCITEPRESS, 2021, pp. 210–221.
35. Kim, J.; Duguma, D.G.; Lee, S.; Kim, B.; Lim, J.; You, I. Scrutinizing the Vulnerability of Ephemeral Diffie–Hellman over COSE (EDHOC) for IoT Environment Using Formal Approaches. *Mobile Information Systems* **2021**, *2021*.
36. Günther, F.; Mukendi, M.I.T. Careful with MAc-then-SIGn: A Computational Analysis of the EDHOC Lightweight Authenticated Key Exchange Protocol. In Proceedings of the EuroS&P 2023. IEEE, 2023, pp. 773–796.
37. Cremers, C.; Dax, A.; Medinger, N. Keeping Up with the KEMs: Stronger Security Notions for KEMs and Automated Analysis of KEM-based Protocols. In Proceedings of the CCS 2024. ACM, 2024, pp. 1046–1060.
38. Cottier, B.; Pointcheval, D. Security Analysis of Improved EDHOC Protocol. In Proceedings of the Foundations and Practice of Security (FPS) 2022. Springer, 2022, pp. 3–18.
39. Ferreira, L. Computational Security Analysis of the Full EDHOC Protocol. In Proceedings of the CT-RSA 2024. Springer, 2024, pp. 25–48.
40. Shin, S.; et al. A Hybrid EDHOC Protocol Post-Quantum Secure against Key Compromise Impersonation. In Proceedings of the Proceedings of eBISION 2026, 2026.

41. IETF Internet-Draft. Hybrid key exchange in TLS 1.3. https://www.ietf.org/archive/id/draft-ietf-tls-hybrid-design-10.html, 2024. Accessed on 21 February 2026.
42. IETF RFC 5295. Specification for the Derivation of Root Keys from an Extended Master Session Key (EMSK). https://www.rfc-editor.org/rfc/rfc5295.html, 2008. Accessed on 5 March 2026.
43. IETF RFC 8613. Object Security for Constrained RESTful Environments (OSCORE). https://www.rfc-editor.org/rfc/rfc8613.html, 2019. Accessed on 5 March 2026.
44. NIST FIPS 203. Module-Lattice-Based Key-Encapsulation Mechanism Standard. https://csrc.nist.gov/pubs/fips/203/final, 2024. Accessed on 2 February 2026.
45. CRYSTALS. https://pq-crystals.org/. Accessed on 10 February 2026.
46. ETSI TS 103 744. CYBER; Quantum-safe Hybrid Key Exchanges. https://www.etsi.org/deliver/etsi_ts/103700_103799/103744/01.01.01_60/ts_103744v010101p.pdf, 2020. Accessed on 12 February 2026.
47. Angel, Y.; Dowling, B.; Hülsing, A.; Schwabe, P.; Weber, F. Post Quantum Noise. In Proceedings of the CCS 2022. ACM, 2022, pp. 97–109.
48. IETF RFC 9370. Multiple Key Exchanges in the Internet Key Exchange Protocol Version 2 (IKEv2). https://www.rfc-editor.org/rfc/rfc9370.html, 2023. Accessed on 11 February 2026.
49. IETF Internet-Draft. Composite ML-KEM for Use in the Internet X.509 Public Key Infrastructure and CMS. https://www.ietf.org/archive/id/draft-ietf-lamps-pq-composite-kem-04.html, 2024. Accessed on 22 February 2026.
50. The PQXDH Key Agreement Protocol. https://signal.org/docs/specifications/pqxdh/. Accessed on 10 February 2026.
51. Fiedler, R.; Günther, F. Security Analysis of Signal's PQXDH Handshake. https://eprint.iacr.org/2024/702, 2024. Cryptology ePrint Archive: Paper 2024/702.
52. Dolev, D.; Yao, A.C. On the Security of Public Key Protocols. *IEEE Transactions on Information Theory* **1983**, *29*, 198–208. https://doi.org/10.1109/TIT.1983.1056650.
53. Krawczyk, H. SIGMA: The 'SIGn-and-MAc' Approach to Authenticated Diffie-Hellman and Its Use in the IKE Protocols. In Proceedings of the CRYPTO 2003. Springer, 2003, pp. 400–425.
54. NIST FIPS 204. Module-Lattice-Based Digital Signature Standard. https://csrc.nist.gov/pubs/fips/204/final, 2024. Accessed on 13 February 2026.
55. Dolev, D.; Yao, A. On the Security of Public Key Protocols. In Proceedings of the IEEE Symposium on Foundations of Computer Science, 1983.
56. Cremers, C.; Horvat, M.; Hoyland, J. A Symbolic Analysis of TLS 1.3. In Proceedings of the IEEE Symposium on Security and Privacy, 2017.
57. ISO/IEC 29128: Information technology – Security techniques – Verification of cryptographic protocols. International Organization for Standardization (ISO), ISO/IEC 29128, 2011.
58. The Libsodium Authors. Libsodium: A modern and easy-to-use crypto library. https://libsodium.org, 2024. Accessed: 2026-04-03.
59. Mbed TLS Contributors. Mbed TLS: Lightweight, modular, and portable cryptographic library. https://github.com/Mbed-TLS/mbedtls, 2024. Accessed: 2026-04-03.
60. PQClean Project Contributors. PQClean: Clean, portable, tested implementations of post-quantum cryptography. https://github.com/PQClean/PQClean, 2024. Accessed: 2026-04-03.
61. Open Quantum Safe Project. liboqs: C library for prototyping and experimenting with quantum-resistant cryptography. https://github.com/open-quantum-safe/liboqs, 2024. Accessed on 5 March 2026.
62. Open Quantum Safe Project. oqs-provider: Open Quantum Safe Provider for OpenSSL 3. https://github.com/open-quantum-safe/oqs-provider, 2024. Accessed on 17 March 2026.
63. IETF RFC 5749. Distribution of EAP-Based Keys for Handover and Re-Authentication. https://www.rfc-editor.org/rfc/rfc5749.html, 2010. Accessed on 5 March 2026.
64. IETF RFC 6696. EAP Extensions for the EAP Re-authentication Protocol (ERP). https://www.rfc-editor.org/rfc/rfc6696.html, 2012. Accessed on 15 August 2026.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.